TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA Công Nghệ Thông Tin
BỘ MÔN: Công Nghệ Phần Mềm

ĐỀ THI VÀ BÀI LÀM

Tên học phần: Math for CS
Mã học phần: Hình thức thi: *Tự luận có giám sát*
Đề số: **01/02** Thời gian làm bài: 90 phút *(không kể thời gian chép/phát đề)*
Được sử dụng tài liệu khi làm bài.

Họ tên: Trịnh Minh Đạt **Lớp**: 23D_DT1 **MSSV**: 102230180

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam:

Câu 1 (4 điểm): Viết chương trình nhập mã số sinh viên của bản thân(N) , thực hiện các công việc sau:

a) Hãy viết hàm để phân tích N ra hữu hạn các số nguyên tố

Trả lời: Dán code vào bên dưới:

```
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def prime_factorize(n):
    primes = []
    exponents = []
    for x in range(2, int(math.sqrt(n)) + 1):
        if is_prime(x):
            if n % x == 0:
                count = 0
                while n % x == 0:
                    n //= x
                    count += 1
                primes.append(x)
                exponents.append(count)
    if n > 1:
        primes.append(n)
        exponents.append(1)
    return primes, exponents

def main():
```

```
    n = int(input("Nhập vào số n: "))

    if is_prime(n):
        print(f"{n} là số nguyên tố!")
    else:
        print(f"{n} không phải là số nguyên tố!")

    primes, exponents = prime_factorize(n)

    print(f"Tích các thừa số nguyên tố của {n} là:")
    print(f"{n} =", end=" ")
    for i in range(len(primes)):
        print(f"{primes[i]}", end="")
        if exponents[i] > 1:
            print(f"^{exponents[i]}", end="")
        if i < len(primes) - 1:
            print(" * ", end="")
    print()

if __name__ == "__main__":
    main()
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhập vào số n: 102230180
102230180 không phải là số nguyên tố!
Tích các thừa số nguyên tố của 102230180 là:
102230180 = 2^2 * 5 * 13 * 17 * 101 * 229
```

b) Áp dụng câu a) hãy viết hàm tính tổng các ước của N

Trả lời: Dán code vào bên dưới

```
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def prime_factorize(n):
    primes = []
    exponents = []
    for x in range(2, int(math.sqrt(n)) + 1):
        if is_prime(x):
            if n % x == 0:
                count = 0
                while n % x == 0:
                    n //= x
                    count += 1
                primes.append(x)
```

```
            exponents.append(count)
    if n > 1:
        primes.append(n)
        exponents.append(1)
    return primes, exponents

def sum_of_divisors(n):
    primes, exponents = prime_factorize(n)
    total_sum = 1

    for i in range(len(primes)):
        prime = primes[i]
        exp = exponents[i]


        sum_for_prime = sum(prime**k for k in range(exp + 1))
        total_sum *= sum_for_prime

    return total_sum

def main():
    n = int(input("Nhập vào số n: "))

    if is_prime(n):
        print(f"{n} là số nguyên tố!")
    else:
        print(f"{n} không phải là số nguyên tố!")

    primes, exponents = prime_factorize(n)

    print(f"Tích các thừa số nguyên tố của {n} là:")
    print(f"{n} =", end=" ")
    for i in range(len(primes)):
        print(f"{primes[i]}", end="")
        if exponents[i] > 1:
            print(f"^{exponents[i]}", end="")
        if i < len(primes) - 1:
            print(" * ", end="")
    print()

    total_sum = sum_of_divisors(n)
    print(f"Tổng các ước của {n} là: {total_sum}")

if __name__ == "__main__":
    main()
```

c) Áp dụng câu a) hãy viết hàm tính tích các ước của N

Trả lời: Dán code vào bên dưới

```
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
```

```
    return True

def prime_factorize(n):
    primes = []
    exponents = []
    for x in range(2, int(math.sqrt(n)) + 1):
        if is_prime(x):
            if n % x == 0:
                count = 0
                while n % x == 0:
                    n //= x
                    count += 1
                primes.append(x)
                exponents.append(count)
    if n > 1:
        primes.append(n)
        exponents.append(1)
    return primes, exponents

def sum_of_divisors(n):
    primes, exponents = prime_factorize(n)
    total_sum = 1

    for i in range(len(primes)):
        prime = primes[i]
        exp = exponents[i]
        sum_for_prime = sum(prime**k for k in range(exp + 1))
        total_sum *= sum_for_prime

    return total_sum

def product_of_divisors(n):
    primes, exponents = prime_factorize(n)

    total_divisors = 1
    for exp in exponents:
        total_divisors *= (exp + 1)

    product = n ** (total_divisors // 2)

    return product

def main():
    n = int(input("Nhập vào số n: "))

    if is_prime(n):
        print(f"{n} là số nguyên tố!")
    else:
        print(f"{n} không phải là số nguyên tố!")

    primes, exponents = prime_factorize(n)

    print(f"Tích các thừa số nguyên tố của {n} là:")
```

```
    print(f"{n} =", end=" ")
    for i in range(len(primes)):
        print(f"{primes[i]}", end="")
        if exponents[i] > 1:
            print(f"^{exponents[i]}", end="")
        if i < len(primes) - 1:
            print(" * ", end="")
    print()

    total_product = product_of_divisors(n)
    print(f"Tích các ước của {n} là: {total_product}")

if __name__ == "__main__":
    main()
```

d) Hãy viết hàm để xác định các số hoàn hảo không vượt quá N

Trả lời: Dán code vào bên dưới

```
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def find_perfect_numbers(limit):
    perfect_count = 1
    p = 2
    while True:
        mersenne_prime = 2**p - 1
        if is_prime(mersenne_prime):
            perfect_number = (2**(p-1)) * mersenne_prime
            if perfect_number > limit:
                break
            print(f"{perfect_count}. {perfect_number} là số hoàn hảo!")
            perfect_count += 1
        p += 1

def main():
    n = int(input("Nhập vào số n: "))
    print(f"Các số hoàn hảo từ 1 đến {n} là:")
    find_perfect_numbers(n)

if __name__ == "__main__":
    main()
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhập vào số n: 102230180
Các số hoàn hảo từ 1 đến 102230180 là:
1. 6 là số hoàn hảo!
2. 28 là số hoàn hảo!
3. 496 là số hoàn hảo!
4. 8128 là số hoàn hảo!
5. 33550336 là số hoàn hảo!
```

Câu 2 (3 *điểm*): Phân rã ma trận A (nếu mã số sinh viên là số lẽ thì thực hiện phân rã Cholesky, còn mã sinh viên là chẵn thì thực hiện phân rã SVD)

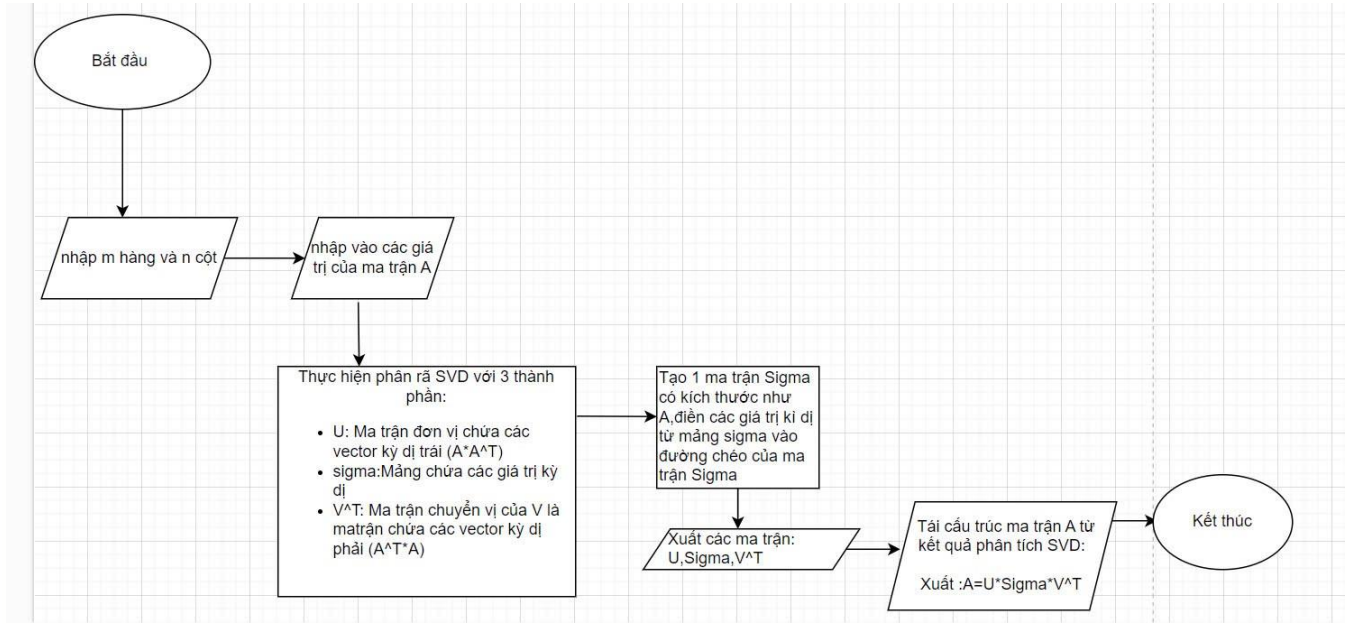
a) Hãy trình bày điều kiện của ma trân A để thực hiện phân rã Cholesky / SVD.

Trả lời: Trình bày điều kiện vào bên dưới:

Không cần điều kiện của ma trận A để thực hiện phân rã SVD.

b) Hãy dùng sơ đồ khối để miêu tả thuật toán phân rã Cholesky / SVD.

Trả lời: Dán sơ đồ khối vào đây vào bên dưới (khuyến khích dùng io draw)



c) Viết hàm để thực thi phân rã Cholesky / SVD.

Trả lời: Dán code vào bên dưới

```
import numpy as np

def svd_decomposition(A):
    """Performs Singular Value Decomposition (SVD) of matrix A."""
    U, sigma, VT = np.linalg.svd(A)

    # Create a diagonal matrix for sigma
    Sigma = np.zeros((A.shape[0], A.shape[1]))
    np.fill_diagonal(Sigma, sigma)
```

```
    return U, Sigma, VT

def main():
    # Input the size of the matrix (rows and columns)
    m = int(input("Enter the number of rows (m): "))
    n = int(input("Enter the number of columns (n): "))

    # Input matrix A from user
    print("Enter matrix A:")
    A = np.zeros((m, n))
    for i in range(m):
        for j in range(n):
            A[i, j] = float(input(f"A[{i+1},{j+1}]: "))

    # Perform SVD decomposition
    U, Sigma, VT = svd_decomposition(A)

    # Display the results
    print("\nMatrix A:")
    print(A)
    print("\nMatrix U (Left Singular Vectors):")
    print(U)
    print("\nDiagonal Matrix Σ (Singular Values):")
    print(Sigma)
    print("\nMatrix V^T (Right Singular Vectors):")
    print(VT)

    # Verify decomposition by reconstructing A
    A_reconstructed = np.dot(U, np.dot(Sigma, VT))
    print("\nReconstructed Matrix A (UΣV^T):")
    print(A_reconstructed)

if __name__ == "__main__":
    main()


# Trả lời: Minh hoạ kết quả vào bên dưới
```

```
Enter the number of rows (m): 2
Enter the number of columns (n): 3
Enter matrix A:
A[1,1]: 6
A[1,2]: 2
A[1,3]: 1
A[2,1]: 2
A[2,2]: 3
A[2,3]: 1

Matrix A:
[[6. 2. 1.]
 [2. 3. 1.]]

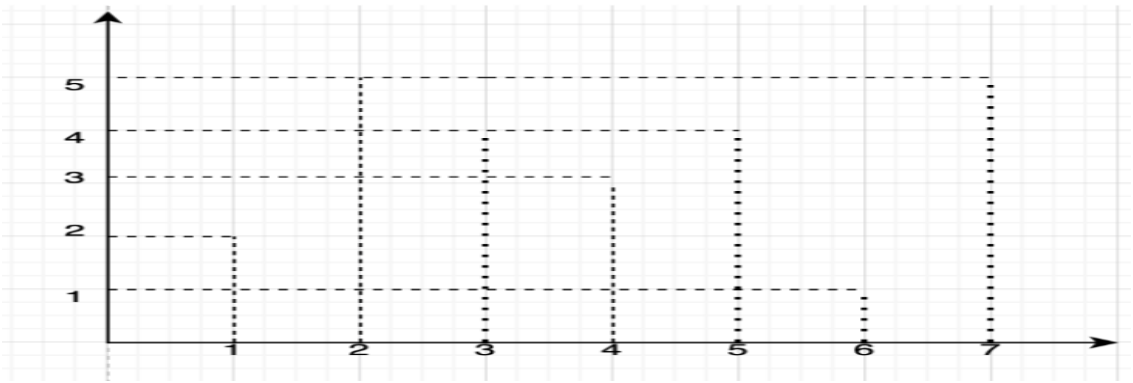
Matrix U (Left Singular Vectors):
[[-0.8885964  -0.45868991]
 [-0.45868991  0.8885964 ]]

Diagonal Matrix Σ (Singular Values):
[[7.12795371 0.         0.        ]
 [0.         2.04750481 0.        ]]

Matrix V^T (Right Singular Vectors):
[[-0.87668334 -0.44237977 -0.18901446]
 [-0.4761633   0.85392199  0.20996605]
 [-0.06851887 -0.27407548  0.95926419]]

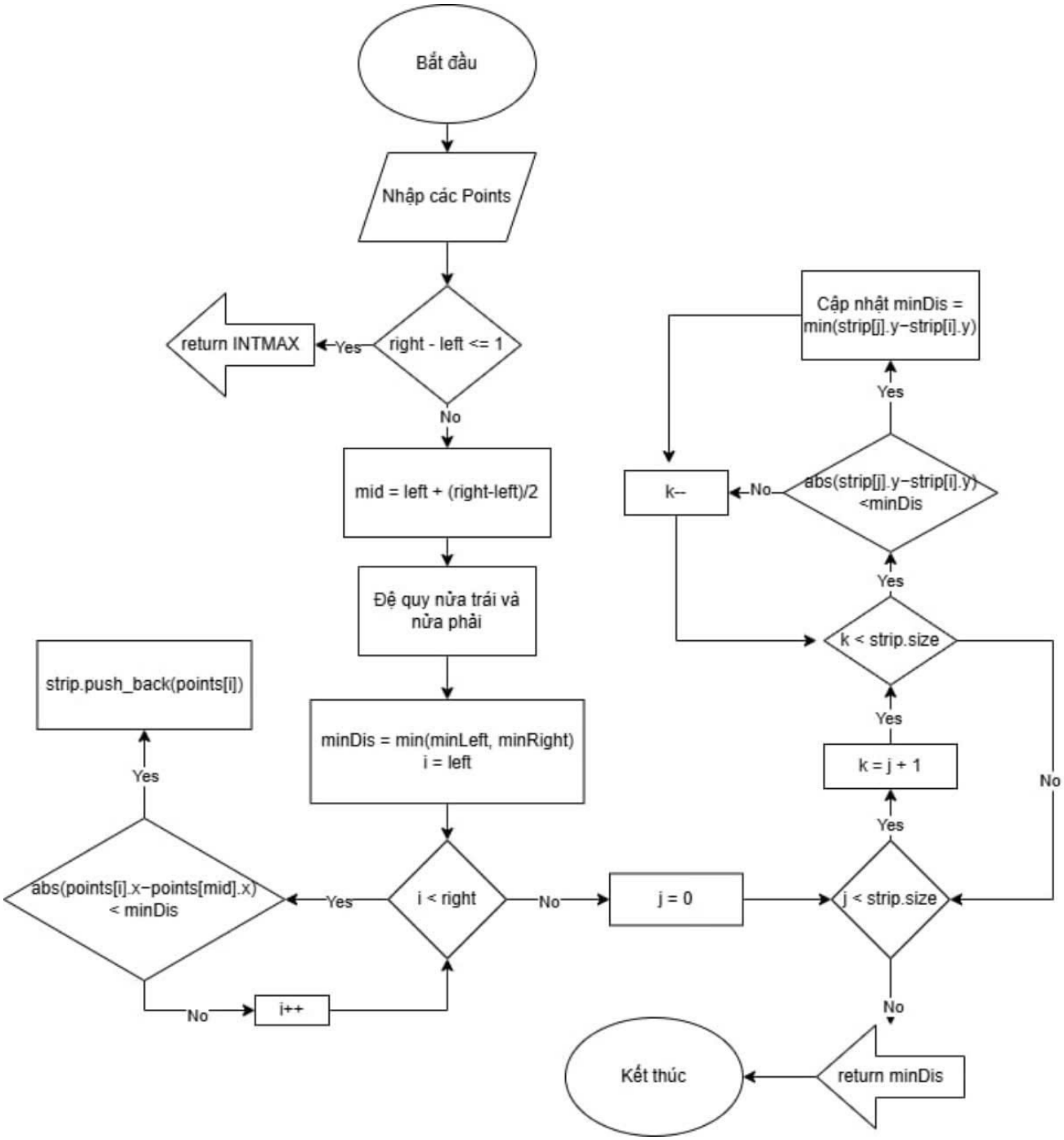
Reconstructed Matrix A (UΣV^T):
[[6. 2. 1.]
 [2. 3. 1.]]
```

Câu 2 (3 *điểm*): Cho không gian Oxy và 7 điểm tương ứng như hình vẽ dưới:



a) *Dùng sơ đồ khối để mô tả thuật toán xác định khoảng cách ngắn nhất giữa 2 điểm trong N điểm(thuật toán đảm bảo nlog(n)*

Trả lời: Dán sơ đồ khối vào bên dưới:



b) *Viết chương trình dạng hàm mô tả thuật toán ở câu a)*

Trả lời: viết câu trả lời vào bên dưới:

```
def manhattan_distance(p1, p2):
    """Tính khoảng cách Manhattan giữa hai điểm p1 và p2."""
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
```

```
def find_min_manhattan_distance(points):
    """Tìm tất cả các cặp điểm có khoảng cách Manhattan ngắn nhất."""
    min_distance = float('inf')
    closest_pairs = []

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            distance = manhattan_distance(points[i], points[j])
            if distance < min_distance:
                min_distance = distance
                closest_pairs = [(points[i], points[j])]
            elif distance == min_distance:
                closest_pairs.append((points[i], points[j]))

    return min_distance, closest_pairs

def main():
    N = int(input("Nhập số lượng điểm N: "))

    points = []
    for i in range(N):
        x = int(input(f"Nhập tọa độ x của điểm {i+1}: "))
        y = int(input(f"Nhập tọa độ y của điểm {i+1}: "))
        points.append((x, y))

    min_distance, closest_pairs = find_min_manhattan_distance(points)
```

```
    print(f"Khoảng cách Manhattan ngắn nhất là {min_distance}.")
    print("Các cặp điểm có khoảng cách ngắn nhất:")
    for p1, p2 in closest_pairs:
        print(f"Điểm {p1} và {p2}")


if __name__ == "__main__":
    main()
```

Trả lời: Dán kết quả minh hoạ 7 điểm đã cho

```
Nhập số lượng điểm N: 7
Nhập tọa độ x của điểm 1: 1
Nhập tọa độ y của điểm 1: 2
Nhập tọa độ x của điểm 2: 2
Nhập tọa độ y của điểm 2: 5
Nhập tọa độ x của điểm 3: 3
Nhập tọa độ y của điểm 3: 4
Nhập tọa độ x của điểm 4: 4
Nhập tọa độ y của điểm 4: 3
Nhập tọa độ x của điểm 5: 5
Nhập tọa độ y của điểm 5: 4
Nhập tọa độ x của điểm 6: 6
Nhập tọa độ y của điểm 6: 1
Nhập tọa độ x của điểm 7: 7
Nhập tọa độ y của điểm 7: 5
Khoảng cách Manhattan ngắn nhất là 2.
Các cặp điểm có khoảng cách ngắn nhất:
Điểm (2, 5) và (3, 4)
Điểm (3, 4) và (4, 3)
Điểm (3, 4) và (5, 4)
Điểm (4, 3) và (5, 4)
```

Đà Nẵng, ngày 04 tháng 10 năm 2024

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI

TRƯỞNG BỘ MÔN
(đã duyệt)